

M4Trust — Ana Yapı ve Kararlar

Moka United Fintech Hackathon · Geliştirme süresi: 5-6 gün · Ekip: Berke (backend + frontend) & Yusuf (AI)

1. Konumlandırma (tek cümle)

M4Trust, KOBİ'lerin sözleşmeye bağlı B2B ödemelerini AI destekli sözleşme okuma, mevzuat destekli RAG, çift taraflı onay ve deterministik ödeme kurallarıyla güvenli hale getiren Moka-ready bir güven ve kanıt katmanıdır. Bağımsız escrow şirketi değil; lisanslı ödeme altyapılarının üstünde çalışan zekâ katmanı.

2. Temel tasarım ilkeleri

- 1 AI önerir, insanlar onaylar, deterministik motor uygular. LLM asla para hareket ettirmez; onaydan sonra para yolundan tamamen çıkar.
- 2 Escrow satmıyoruz, karar ve kanıt katmanı kuruyoruz. Demo uygulamada hash zorunlu değil; production düşünülürse zaman damgalı evidence package ve bütünlük kontrolleri eklenebilir.
- 3 Kademeli otonomi. "Suistimal imkânsız" değil; "anlaşmazlıklar daha nadir, daha küçük, daha kanıtlı."
- 4 Türkiye avantajı: GİB e-fatura/e-irsaliye = devlet kalitesinde teslimat oracle'ı. Video analizi ana kanıt değil, ikincil doğrulama ve risk sinyali.
- 5 Hedef segment: Akreditife erişemeyen %99 — ilk kez ticaret yapan KOBİ'ler, mikro ihracatçılar.
- 6 Dürüst mühendislik dili: "Smart contract" yok; "deterministik kural motoru" var.
- 7 Local-first, external-minimal. Dış API'ler ana yol değil; runtime'da sadece 5.4 mini API ana LLM olarak kullanılır. OCR, doküman parsing, validator, demo ödeme ve video analizi lokal çalışır.

3. Mimari kararı

Tek FastAPI servisi + React frontend + SQLite. Spring hibriti ve PostgreSQL bu demo için gereksiz entegrasyon maliyeti doğurduğu için elendi.

- Backend: Python 3.12, FastAPI. AI pipeline, RAG, validator, demo state machine, mock ödeme adapter'ı ve evidence üretimi aynı serviste, modül sınırlarıyla ayrılmış.
- Frontend: React + Vite + Tailwind. Dört ekran: upload → contract diff (madde highlight + çıkarılan kural + confidence) → çift taraflı onay → dashboard (durum, dispute, evidence indirme).
- DB: SQLite. Bu proje demo uygulama olduğu için taşınabilirlik ve hızlı kurulum, gerçekçi altyapı görüntüsünden daha öncelikli.
- Demo ortamı: Lokal makine ana yol. Bulut yedek deploy planlanmıyor. Sistem hafif tutulacağı için demo sözleşmelerinin hash eşleşmesiyle cache'lenmesine gerek yok.

4. Tech stack kararları

Katman	Karar	Not
PDF parser	PyMuPDF / PyMuPDF4LLM	Dijital PDF'de OCR'a girmeden metin, tablo ve temel markdown çıkarır. Ana yol

Katman	Karar	Not
DOCX parser	python-docx veya mammoth	DOCX'te OCR yok; doğrudan text/structure extraction. MVP'de PDF öncelikli, DOCX sade destek
OCR	Tesseract baseline	Temiz scan/görsel sayfalarda CPU ile çalışır, Türkçe destekler, dependency az. Doğrudan markdown üretmez; markdown post-process katmanı bizde
OCR mimarisi	DocumentExtractor interface	1) DigitalPDFExtractor 2) DocxExtractor 3) TesseractOCR 4) MarkdownNormalizer. Marker/Surya/Datalab/Chandra ağır fallback'i MVP'den kaldırıldı
LLM	5.4 mini API	Ana extraction/reasoning modeli. Structured output (JSON schema) zorunlu. LLM öneri üretir; validator geçmeden kural olamaz
Açık kaynak NLP benchmark	BERTurk + Qwen/DeepSeek/Llama ailesinden açık ağırlık adaylar	Colab'da "mümkün ama kaynak/zaman lazım mı?" sorusunu göstermek için ar-ge deneyi
RAG embedding	BAAI/bge-m3 + ChromaDB	Mevzuat korpusundan ilgili parçaları bulur; LLM'e kaynaklı context döner. BGE-M3 karar vermez, yalnızca retrieval/embedding katmanıdır
RAG korpusu	6493, 5549, 6098 TBK, KVKK	Kanun ve mevzuat parçaları chunk'lanır, embed edilir ve lokal Chroma index olarak kullanılır
Teslimat video analizi	Lightweight detector + frame sampling/tracking	Gelen mallar artık tek görselden değil videodan değerlendirilecek. Koli/palet/damaged_box sayımı risk sinyali üretir
Ödeme	Mock Moka Adapter	Pre-auth/hold, capture, partial capture, void/refund benzeri state geçişleri simüle edilir
Evidence	Timestamp + JSON bundle	Sözleşme markdown'u, extracted rule JSON, onaylar, teslimat event'leri, validator gerekçesi, ödeme kararı

5. Uçtan uca akış

PDF/DOCX/görsel sözleşme yükle → doküman triage → dijital PDF ise PyMuPDF ile metin/markdown çıkar → DOCX ise doğrudan text/structure extraction → scan/görsel ise Tesseract OCR → her durumda modele gidecek nihai sözleşme formatı markdown → BGE-M3/Chroma ile mevzuat context'i getir → 5.4 mini structured extraction + reasoning → deterministik validator → contract diff ve kural özeti → çift taraflı onay → onaydan sonra LLM para yolundan çıkar → mock ödeme bekletme/pre-auth → teslimat kanıtları (e-irsaliye birincil, video analizi ikincil) → decision engine → capture/partial/dispute/hold sonucu.

BGE-M3'ün amacı net: RAG için kaynak taraması yapar. Sözleşme veya kural taslağı geldiğinde ilgili kanun/mevzuat chunk'larını bulur, LLM'e "kaynaklı bağlam" olarak verir. Analizi LLM yapar, kontrolü validator yapar. BGE-M3 tek başına hukuki yorum veya ödeme kararı vermez.

Decision engine 4 sonucu:

- Tamamlandı → Capture (tam ödeme simülasyonu)
- Kısmi teslim → Partial capture (oransal ödeme simülasyonu)

- Beklemede → Para bekletme/pre-auth devam
- Çelişkili kanıt → Dispute kaydı + evidence snapshot; ödeme adapter'ına capture gitmez

6. Validator katmanı

Validator, LLM çıktısını "güzel görünen metin" olmaktan çıkarıp sistemin çalıştırabileceği güvenli kural setine çeviren deterministik kapıdır. LLM çıktısı validator'dan geçmeden veritabanına aktif ödeme kuralı olarak yazılmaz.

Kontrol	Ne yapar?
JSON schema kontrolü	Zorunlu alanlar, enum değerleri, tarih/para/yüzde tipleri doğru mu?
Matematik kontrolü	Milestone yüzdeleri toplamı 100 mü, partial capture oranı negatif veya 100 üstü mü?
Tarih mantığı	Sözleşme tarihi, teslim tarihi, ödeme vadesi ve onay zamanı çelişiyor mu?
Taraf kontrolü	Alıcı/satıcı/vergi no/şirket adı sözleşme içinde tutarlı mı?
Kaynak kontrolü	LLM'in çıkardığı kritik kuralın sözleşmede ve varsa RAG context'inde dayanağı var mı?
Belirsizlik kontrolü	Confidence düşükse veya kaynak cümle eksikse otomatik onay yerine manual review üretir
State machine kontrolü	Onay alınmadan pre-auth/capture simülasyonu çalışmaz; dispute durumunda ödeme aksiyonu kilitlenir
Teslimat kontrolü	E-irsaliye miktarı, sözleşme miktarı ve video sayım sinyali çelişirse dispute/manual review açar

Validator çıktısı üç sınıftır: PASS, NEEDS_REVIEW, REJECT. UI'da sadece "geçti/geçmedi" değil, kısa gerekçe de gösterilir. Altın demo anı, yüzdeleri 100 etmeyen bozuk sözleşmenin validator tarafından reddedilmesidir.

7. Teslimat video analizi

Gelen mallar artık tek görselden değil, kısa video kanıtından değerlendirilecek. Video tarafı ödeme kararının tek kaynağı değildir; e-irsaliye ve sözleşme verisiyle birlikte ikincil doğrulama sinyali üretir.

Önerilen MVP yaklaşımı:

- Videodan belirli aralıklarla frame alınır.
- Lightweight detector koli/palet/damaged_box gibi sınıfları tespit eder.
- Basit tracking/dedup mantığıyla aynı kolinin tekrar sayılması azaltılır.
- Çıktı delivery_video_analyzed event'i olarak decision engine'e gider.

Mentor notu: Moka mentorlarına özellikle video kanıtının yanıltıcı açıkları danişılacak. Örneğin 3x3'lük koli küpünde ortadaki kolon boşaltılmış olabilir; video dışarıdan hâlâ tam teslimat gibi görünebilir. Bu açıkları kapatmak için çok açıdan video, yakın plan tur, rastgele kutu açılımı, ağırlık bilgisi, barkod/etiket eşleşmesi ve e-irsaliye çapraz kontrolü gibi önlemlerin production'da nasıl ele alınacağı sorulacak.

8. Ödeme ve Moka Adapter notu

Demo için MockMokaAdapter yeterli olacak. Kısıtlı zamanda gerçek ödeme entegrasyonu yerine state transition simülasyonu yapılacaktır:

- reserve_or_preauth
- capture
- partial_capture
- void_or_release
- mark_dispute

Mentor notu: Moka United mentorlarına iki taraf onayına kadar parayı bekletme/pre-auth/hold benzeri yapının mümkün olup olmadığı, banka hesapları arası transferlerin nasıl işlediği, Moka'nın bunu doğrudan sağlayıp sağlayamayacağı veya production seviyede hangi lisans/partner yapısının gerektiği sorulacak.

9. Açık kaynak model benchmark planı

Production/demo LLM kararı 5.4 mini API. Buna ek olarak ar-ge değeri göstermek için Colab'da açık kaynak NLP/LLM benchmark'ı yapılacak. Amaç 5.4 mini'yi hemen değiştirmek değil; "bu görev açık kaynak modellerle mümkün mü, yoksa ciddi kaynak/zaman mı gerektirir?" sorusuna kanıt üretmek.

Aday	Statü / yorum	Benchmark rolü
BERTurk (dbmdz/bert-base-turkish-cased)	Açık, Türkçe odaklı encoder	Field extraction'ı NER/sınıflandırma olarak dener; küçük ve akademik ar-ge değeri yüksek
Qwen3-14B / Qwen3-32B	Qwen Plus/Max API-only olabilir; açık ağırlık karşılığı Qwen3 ailesi	Colab'da quantized veya uygun GPU ile structured extraction karşılaştırması
Qwen3-235B-A22B	Açık ağırlık, güçlü ama Colab için pratik değil	"Kaynak varsa mümkün" argümanı için referans; self-host MVP hedefi değil
DeepSeek-R1-Distill-Qwen-14B/32B veya DeepSeek V4 Flash/Pro erişilebilen açık ağırlık varyantı	V4 Pro kapasite olarak güçlü ama self-host maliyeti çok yüksek olabilir	Uygun olan küçük/orta varyant Colab'da denir; büyük varyant kaynak gereksinimi argümanına yazılır
Llama 3.1 8B / Llama 3.3 70B / Llama 4 Scout-Maverick erişim durumuna göre	Llama ailesi açık ağırlık/Community License hattı	Llama 3.1 8B pratik Colab baseline; daha büyük Llama varyantları kaynak gereksinimi notu

Benchmark yöntemi:

- 1 20-30 sözleşme parçası veya sentetik Türkçe ticari madde hazırlanır.
- 2 Her örnek için altın extraction_json elle düzeltilir.
- 3 Aynı markdown sözleşme metni ve aynı RAG context'i tüm modellere verilir.
- 4 Çıktılar aynı JSON schema'ya zorlanır.
- 5 Metrikler: field-level F1, tutar/yüzde exact match, tarih doğruluğu, validator pass rate, kaynak cümle doğruluğu, latency, VRAM/compute ihtiyacı.
- 6 Sonuç üç cümleye indirgenir: "mümkün ve hafif", "mümkün ama veri/kaynak lazım", veya "MVP için hosted LLM daha mantıklı".

10. Extraction JSON şeması önerisi

LLM'in üreteceği çıktı tek bir sabit şemaya zorlanmalı. Önerilen minimum şema:

```
{
  "contract_id": "string",
  "parties": {
    "buyer": {"name": "string", "tax_id": "string|null"},
    "seller": {"name": "string", "tax_id": "string|null"}
  },
}
```

```

"commercial_terms": {
  "currency": "TRY|USD|EUR|OTHER",
  "total_amount": 0,
  "goods": [{"name": "string", "quantity": 0, "unit": "string"}],
  "delivery_deadline": "YYYY-MM-DD|null"
},
"payment_rules": [
  {
    "milestone": "string",
    "trigger": "approval|e_invoice|delivery_video|manual_review",
    "percentage": 0,
    "required_evidence": ["contract", "e_irsaliye", "video"],
    "source_quote": "string",
    "confidence": 0.0
  }
],
"risk_flags": ["string"],
"needs_manual_review": false
}

```

Validator bu şemayı okur, eksik/çelişkili alanları reddeder veya manual review'a yollar.

11. İç event/webhook şeması ne demek?

Bu projede gerçek dış webhook kurmak şart değil. "İç event/webhook şeması" denilen şey, tek FastAPI servisinin modülleri arasında dolaşacak standart JSON olaylarıdır. Amaç, e-irsaliye simülasyonu, video analizi, kullanıcı onayı ve ödeme adapter'ının aynı dili konuşmasıdır.

Örnek event'ler:

- contract_extracted
- rules_validated
- buyer_approved
- seller_approved
- e_irsaliye_received
- delivery_video_analyzed
- payment_decision_created
- mock_payment_executed
- dispute_opened

Her event minimum transaction_id, event_type, payload, created_at ve source alanlarını taşır.

12. E-irsaliye simülasyonu

Gerçek GiB entegrasyonu yapılmayacak. Demo için e-irsaliye, dashboard'da bir butonla veya basit bir endpoint ile sisteme düşen JSON event'i olarak simüle edilecek.

Önerilen pratik yol:

- Dashboard'da "e-irsaliye geldi" butonu.
- Buton POST /demo/e-irsaliye endpoint'ine hazır JSON gönderir.
- JSON içinde irsaliye numarası, alıcı/satıcı, ürün kalemleri, miktar, teslim tarihi ve durum olur.
- Decision engine bu event'i sözleşme kuralı ve video event'i ile karşılaştırır.

13. Yapılacak iş parçacıkları

- ☐ Extraction JSON schema'yı backend sabiti olarak tanımla.
- ☐ SQLite şemasını kur: transactions, extracted_rules, approvals, events, evidence, mock_payments.
- ☐ PyMuPDF/PyMuPDF4LLM ile PDF → markdown extraction yaz.

- ☐ DOCX için sade text/markdown extraction yaz.
- ☐ Tesseract ile scan/görsel → text extraction yaz.
- ☐ Tüm sözleşme girdilerini modele gitmeden önce normalize edilmiş markdown'a çevir.
- ☐ BGE-M3 ile mevzuat chunk embedding ve Chroma index hazırlığı yap.
- ☐ RAG retrieval endpoint/fonksiyonunu yaz.
- ☐ 5.4 mini adapter ile structured extraction promptunu yaz.
- ☐ Validator katmanını schema, matematik, tarih, taraf, kaynak ve state kontrolleriyle uygula.
- ☐ Contract diff/kural özeti ekranını hazırla.
- ☐ Çift taraflı onay akışını yaz.
- ☐ Mock Moka Adapter state geçişlerini yaz.
- ☐ E-irsaliye simülasyon endpoint'i ve dashboard butonunu ekle.
- ☐ Video upload/analiz demo akışını ve delivery video event'ini ekle.
- ☐ Dispute/manual review durumunu UI'da göster.
- ☐ Evidence JSON bundle indirme özelliğini ekle.
- ☐ Açık kaynak model benchmark notebook planını ve küçük veri setini hazırla.
- ☐ Demo provası, kırma testleri ve ekran kaydı fallback'ini hazırla.

14. Jüri savunma cümleleri

- 6493 / lisans: "Fonlar lisanslı bir EMI/BaaS partnerinde veya Moka'nın sağlayabileceği uygun ödeme yapısında tutulur; biz bunun üzerindeki zekâ ve karar katmanıyız. Production uygunluğunu Moka mentorlarıyla netleştireceğiz."
- Halüsinasyon: "LLM önerir, validator karar verir, insanlar onaylar — LLM para yolunda değil."
- OCR/dependency: "Her belgeyi büyük modele atmıyoruz. Önce dijital metni local çıkarıyoruz; OCR sadece gerekli sayfalarda çalışıyor. Çıkan nihai metin markdown olarak modele giriyor."
- RAG: "BGE-M3 karar vermez; ilgili mevzuat parçalarını bulur. LLM bu kaynaklarla analiz eder, validator da deterministik kontrol yapar."
- Video kalite/miktar açığı: "Video tek başına kesin ispat değil; e-irsaliye, sözleşme ve gerektiğinde insan kontrolüyle birlikte risk sinyali üretir."
- Neden şimdi/burada: "GİB e-irsaliye = devlet kalitesinde, makine tarafından okunabilir teslimat oracle'ı. Bunu icat etmiyoruz, buna bağlanıyoruz."